32

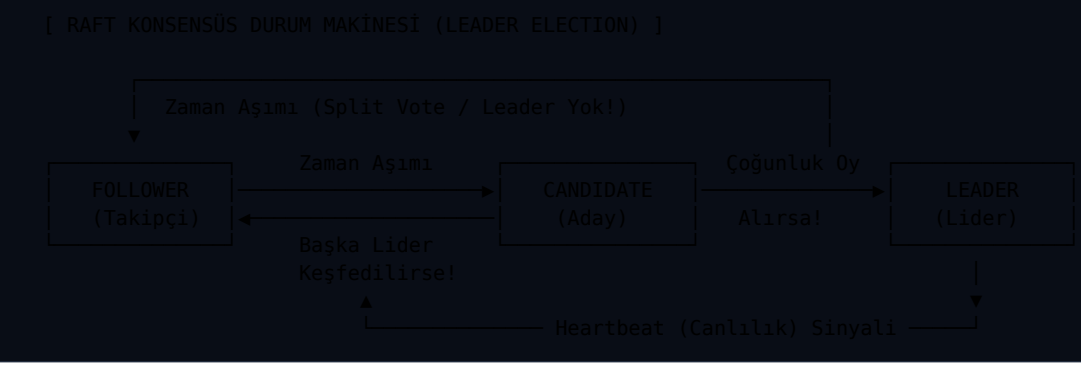
BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Fikir Birliği (Consensus): Raft Algoritması (Lider Seçimi ve Replikasyon)

Distributed Consensus: The Raft Algorithm (Leader Election & Logs)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

5 kişilik bir arkadaş grubunun tatile nereye gideceğine karar vermesini düşünün. Herkes aynı anda konuşursa kaos çıkar. Raft algoritması şudur: 1) İçlerinden biri 'Ben grup lideri olmak istiyorum, bana oy verin' der (Candidate). 2) Eğer arkadaşlarının çoğunluğu (en az 3 kişi) 'Tamam, sen lidersin' derse o kişi Lider olur. 3) Artık tüm kararları lider alır, gruba söyler ve çoğunluk onaylayınca deftere yazar. Lider aniden bayılırsa (Crash), arkadaşlarından biri 'Liderden 150 milisaniyedir ses çıkmıyor' diyerek yeni bir seçim başlatır!

🇬🇧 ENGLISH (ELI5)

Imagine 5 friends voting on a vacation spot. If everyone screams conflicting orders, total chaos erupts. Raft brings democratic order: 1) A member proposes: 'I volunteer as Leader, vote for me!' (Candidate). 2) If a majority quorum (at least 3 out of 5) grants their vote, they become the undisputed Leader. 3) The Leader proposes decisions, replicates the entry to nodes, and commits once a majority confirms. If the Leader faints (crashes), an election timer expires and nodes elect a fresh leader within milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Raft (Ongaro & Ousterhout, Stanford 2014), anlaşılması imkansız derecede karmaşık olan Paxos algoritmasının yerine geliştirilmiş, anlaşılabilir bir konsensüs algoritmasıdır (etcd, Kubernetes, Kafka KRaft, TiDB tarafından kullanılır). 3 temel alt problemden oluşur: 1) Lider Seçimi (Leader Election): Liderden belirli bir süre (Heartbeat timeout: 150-300ms) kalp atışı alamayan Follower, rastgele bir seçim zaman aşımıyla (Randomized Election Timeout) Candidate olur ve `RequestVote` fırlatır. Çoğunluk oyu alan ($N/2 + 1$) lider seçilir. 2) Kütük Replikasyonu (Log Replication): Lider istemciden komutu alır, kendi kütüğüne yazar, follower'lara `AppendEntries` gönderir. Düğümlerin çoğunluğu diske yazdığında işlem 'Committed' kabul edilir. 3) Güvenlik (Safety): Yeni seçilen lider, kütüğü en güncel olan düğüm olmak zorundadır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Raft decomposes consensus into three decomposed state machines: 1) Leader Election: Followers transition to Candidates when heartbeat deadlines (150-300ms randomized timer) expire without leader contact. A candidate requests votes; securing a strict majority quorum ($(N/2) + 1$) crowns the Leader. 2) Log Replication: The Leader appends entries locally, broadcasting `AppendEntries` RPCs. When an entry is replicated across a majority, the Leader commits it and applies to the state machine. 3) Election Safety: A follower will deny vote if the candidate's log is less up-to-date than its own, guaranteeing committed entries are never overwritten or lost. Powers etcd, Kubernetes control planes, and Kafka KRaft.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Raft Rastgele Seçim Zaman Aşımı (Randomized Election Timeout)
import java.util.concurrent.*;

public class RaftElectionTimer {
    public enum Role { FOLLOWER, CANDIDATE, LEADER }

    private Role currentRole = Role.FOLLOWER;
    private long currentTerm = 0;

    public synchronized void onHeartbeatTimeout() {
        if (currentRole != Role.LEADER) {
            currentRole = Role.CANDIDATE;
            currentTerm++;
            int randomTimeout = ThreadLocalRandom.current().nextInt(150, 300);
            IO.println("Term #" + currentTerm + ": Lider düştü! Aday olundu (Timeout: " +
randomTimeout + "ms)");
        }
    }

    public synchronized void becomeLeader() {
        currentRole = Role.LEADER;
        IO.println("🏆 DÜĞÜM LİDER SEÇİLDİ! Term #" + currentTerm);
    }
}

void main() {
    var node = new RaftElectionTimer();
    node.onHeartbeatTimeout();
    node.becomeLeader();
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Raft kümesindeki düğüm sayısı daima TEK SAYI (Odd Number: 3, 5 veya 7) olmalıdır! 4 düğümlü bir kümede 2'ye 2 bölünme (Split Vote) yaşanır ve çoğunluk ($4/2 + 1 = 3$) sağlanamayabilir. 3 düğüm 1 hatayı, 5 düğüm 2 hatayı tolere eder.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Always size Raft clusters with an ODD number of nodes (3, 5, or 7). An even count (4 nodes) offers zero additional fault tolerance over 3 nodes (both tolerate only 1 failure) while elevating split-vote deadlocks.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Raft, dağıtık sistemlerde lider seçimi ve veri replikasyonunu güvenceye alır.
- Rastgele seçim zaman aşımları (Randomized Timeouts) oyların bölünmesini (Split Vote) önler.
- Kubernetes (etcd) ve Kafka (KRaft) fikir birliği omurgasını Raft üzerine kurmuştur.

🎯 Key Takeaways for the Interview

- Raft guarantees leader election and deterministic log replication across distributed nodes.
- Randomized election timeouts disperse candidacy announcements, avoiding voting ties.
- Kubernetes (via etcd) and Kafka KRaft rely directly on Raft for operational consensus.